```
(*
                            CS 51 Lab 19
                  Greenberg-Hastings Cellular Automaton
 https://en.wikipedia.org/wiki/Greenberg-Hastings_cellular_automaton

Implementation of the Greenberg-Hastings model described in the
article

    Greenberg, J. M., and S. P. Hastings. "Spatial Patterns for
    Discrete Models of Diffusion in Excitable Media." SIAM Journal on
    Applied Mathematics 34, no. 3 (1978): 515-23.
    http://www.jstor.org/stable/2100950.
 *)

(*
    ************************************************************
    WARNING: If you have photosensitive epilepsy, you should avoid
    viewing the Greenberg-Hastings cellular automaton. It exhibits
    rapidly flashing visual patterns.
    ************************************************************
 *)

module G = Graphics ;;

(* Automaton parameters *)
let cGRID_SIZE = 100 ;;       (* width and height of grid in cells *)
let cSPARSITY = 25 ;;         (* inverse of proportion of cells initially live *)

(* Rendering parameters *)
let cCOLOR_LEGEND = G.rgb 173 106 108 ;;  (* color for textual legend *)
let cSIDE = 8 ;;              (* width and height of cells in pixels *)
let cRENDER_FREQUENCY = 1     (* how frequently grid is rendered (in ticks) *) ;;
let cFONT = Some "-adobe-times-bold-r-normal--34-240-100-100-p-177-iso8859-9"

type gh_state =
  | Resting
  | Excited
  | Refractory

(* offset index off -- Returns the `index` offset by `off` allowing
   for wraparound. *)
let rec offset (index : int) (off : int) : int =
  if index + off < 0 then
    cGRID_SIZE - (offset ~-index ~-off)
  else
    (index + off) mod cGRID_SIZE ;;

(* gh_update grid i j -- Returns the updated state for cell at i, j
   based on the Greenberg-Hastings excitable media rules. *)
let gh_update (grid : gh_state array array) (i : int) (j : int) =
  let neighbors = ref 0 in
  for i' = ~-1 to ~+1 do
```

```
    for j' = ~-1 to ~+1 do
      (* only consider the four neighbors sharing an index *)
      if (i' = 0 || j' = 0) && not (i' = 0 && j' = 0) then
        let i_ind = offset i i' in
        let j_ind = offset j j' in
        neighbors := !neighbors
                     + match grid.(i_ind).(j_ind) with
                       | Resting -> 0
                       | Refractory -> 0
                       | Excited -> 1
    done
  done;
  match grid.(i).(j) with
  | Excited -> Refractory
  | Refractory -> Resting
  | Resting -> if !neighbors > 0 then Excited else Resting ;;

(* gh_random_state () -- Returns a random cell state. *)
let gh_random_state () =
  let states = [Excited; Refractory; Resting; Resting; Resting; Resting] in
  List.nth states (Random.int (List.length states))

module GHSpec : (Cellular.AUT_SPEC
                 with type state = gh_state) =
  struct
    type state = gh_state
    let name : string = "Greenberg-Hastings"
    let initial : state = Resting
    let grid_size : int = cGRID_SIZE
    let random_state = gh_random_state
    let update = gh_update
    let cell_color (cell : state) : G.color =
      match cell with
      | Resting -> G.rgb 245 240 246
      | Excited -> G.rgb 56 95 113
      | Refractory -> G.rgb 43 65 98
    let side_size : int = cSIDE
    let legend_color : G.color = cCOLOR_LEGEND
    let font : string option = cFONT
    let render_frequency : int = cRENDER_FREQUENCY
  end ;;

module Aut = Cellular.Automaton (GHSpec) ;;

(*.....................................................................
  Running the automaton and displaying the results
 *)

(* main seed -- Runs the automaton using the provided random `seed`
   (for replicability), displaying updates to the graphics window. *)
let main seed =

  Random.init seed;      (* initialize randomness *)
```

```
106   Aut.graphics_init ();  (* ... and graphics window *)
107
108   (* Initialize the grid to a simple half-line of excited and
109   refractory cells, as per the discussion in the article cited at the
110   top of the file. This generates the typical spiral artifacts
111   described in the paper. *)
112   let initial_grid = Aut.fresh_grid () in
113   for i = cGRID_SIZE / 2 to cGRID_SIZE - 1 do
114     initial_grid.(i).(cGRID_SIZE / 2) <- Refractory;
115     initial_grid.(i).(cGRID_SIZE / 2 + 1) <- Excited
116   done;
117
118   Aut.run_grid initial_grid ;;
119
120 (* Run the automaton with user-supplied seed *)
121 let _ =
122   if Array.length Sys.argv <= 1 then
123     Printf.printf "Usage: %s <seed>\n  where <seed> is integer seed\n"
124                   Sys.argv.(0)
125   else
126     main (int_of_string Sys.argv.(1)) ;;
127
```